

Mobile Application Source Code Review

KRPUNE Native, Static Review Findings

Project / App Name	KRPUNE Native (krpune-native)	Version Reviewed	1.0.0
Platform	Android and iOS (Expo / React Native)	Review Date	16 July 2026
Reviewing Team	Fute Services, internal review (not a certified third-party pentest firm)		

1. Purpose and Scope

Static source code review of the krpune-native repository, an offline showcase and tour application for a real estate project covering floor plans, amenities, VR and panorama tours, and location information. The review covered the full application source tree, package dependencies, and configuration files. This is a code-level review only, not a dynamic or runtime penetration test.

2. Standards Used

- ☒ OWASP MASVS
- ☐ OWASP Top 10
- ☐ CERT Secure Coding

3. Methodology

Manual review of source files, combined with automated dependency vulnerability scanning (npm audit). This review does not include dynamic testing, meaning running the application on a device or emulator and intercepting live traffic. That would require a proper mobile VAPT engagement with a qualified testing team.

4. Findings Summary

#	Issue	Location	Severity	Status
1	Permissive WebView configuration (broad file and origin access enabled)	PanoramaViewer.tsx, VR.tsx	Medium	Open
2	11 moderate-severity advisories in transitive build dependencies	package.json (via Expo tooling)	Medium	Open
3	No local storage, no PII, no authentication, no network calls in normal use	Application-wide	Info	N/A

5. Detailed Findings

Finding 1: Permissive WebView configuration

Description: Two screens load locally generated HTML into a WebView with broad file access and universal access settings enabled, an unrestricted origin whitelist, and mixed content always allowed. This is a well-recognised risky combination for Android WebViews.

Impact: Today the WebView only ever loads HTML the application itself writes into its own cache directory, and no remote or user-supplied content reaches it, so practical risk in the current build is low. The risk would become serious only if a future change allowed this WebView to load remote or user-influenced content.

Recommendation: Narrow the WebView settings to only what is actually needed, restrict the origin whitelist, and document why file access is required so the setting is not carried forward unexamined if remote content is ever introduced.

Finding 2: Known vulnerabilities in dependency tree

Description: An automated scan reported 11 moderate-severity advisories rooted in outdated build-tooling packages, pulled in transitively through the Expo CLI and configuration toolchain, not runtime application code.

Impact: These affect the build pipeline rather than the compiled application that ships to end users.

Recommendation: Apply available automated fixes where possible; a full resolution requires a version upgrade that should be scheduled and tested separately as a breaking change.

6. Secure Coding Practices Reviewed

- ☑ Hardcoded secrets / API keys, none found
- ☑ Insecure data storage, no local storage usage found; application stores no user data
- ☑ Insecure network communication, not applicable, application makes no network calls in normal use
- ☑ Third-party library vulnerabilities, see Finding 2

7. Conclusion

This is a self-contained, fully offline showcase application with no user accounts, no PII, and no network activity during normal use. Everything, including images and video, ships bundled inside the installed application. The two findings above are low urgency given the current design, but worth addressing as good practice, particularly before any future change that adds remote or user-supplied content to the in-app WebView. This report is a static review only. If a certified mobile VAPT with dynamic testing is required, that should be commissioned separately with a qualified testing vendor.